

Open Ended lab:2

Group Member: Muhammad Aman (30), Mushaf Khan (43)

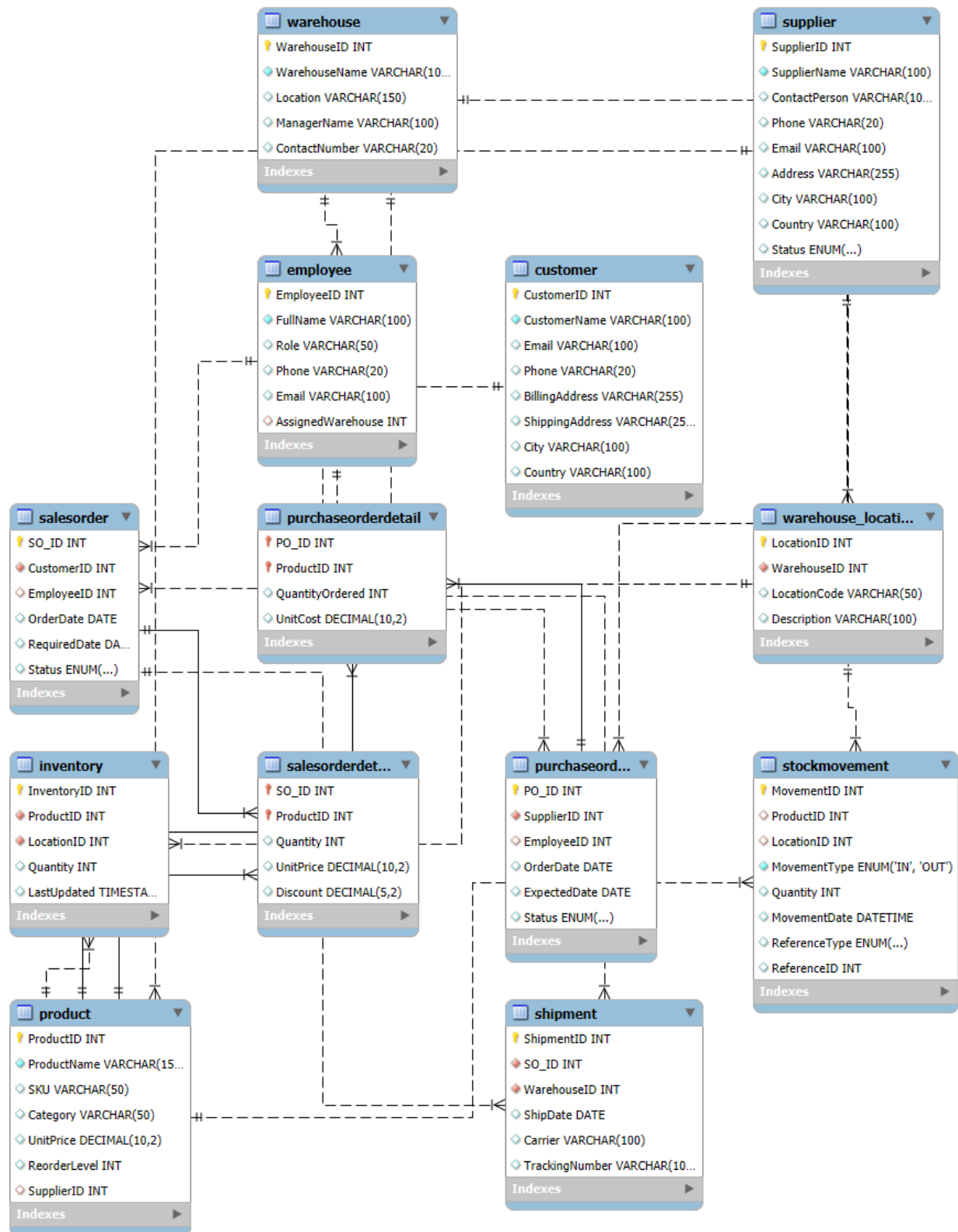
Topic: Warehouse Management System

Task 1(Database Design):

Schema Implementation

```
1 -- 1. Supplier Table
2
3 CREATE TABLE Supplier (
4     SupplierID INT IDENTITY(1,1) PRIMARY KEY,
5     SupplierName VARCHAR(100) NOT NULL,
6     ContactName VARCHAR(100),
7     Phone VARCHAR(20),
8     Email VARCHAR(100),
9     City VARCHAR(50)
10 );
11
12
13 -- 2. Product Table
14
15 CREATE TABLE Product (
16     ProductID INT IDENTITY(1,1) PRIMARY KEY,
17     ProductName VARCHAR(100) NOT NULL,
18     Category VARCHAR(50),
19     UnitPrice DECIMAL(10,2),
20     QuantityPerUnit VARCHAR(50),
21     SupplierID INT,
22     FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)
23 );
24
25 -- 3. Warehouse Table
26
27 CREATE TABLE Warehouse (
28     WarehouseID INT IDENTITY(1,1) PRIMARY KEY,
29     WarehouseName VARCHAR(100) NOT NULL,
30     Location VARCHAR(100),
31     Capacity INT
32 );
33
34 -- 4. Inventory Table
35
36 CREATE TABLE Inventory (
37     InventoryID INT IDENTITY(1,1) PRIMARY KEY,
38     ProductID INT,
39     WarehouseID INT,
40     QuantityInStock INT,
41     LastUpdated DATETIME DEFAULT GETDATE(),
42     FOREIGN KEY (ProductID) REFERENCES Product(ProductID),
43     FOREIGN KEY (WarehouseID) REFERENCES Warehouse(WarehouseID)
44 );
45
46 -- 5. Customer Table
47
48 CREATE TABLE Customer (
49     CustomerID INT IDENTITY(1,1) PRIMARY KEY,
50     CustomerName VARCHAR(100) NOT NULL,
51     Phone VARCHAR(20),
52     Email VARCHAR(100),
53     City VARCHAR(50)
54 );
55
56 -- 6. Orders Table
57
58 CREATE TABLE Orders (
59     OrderID INT IDENTITY(1,1) PRIMARY KEY,
60     CustomerID INT,
61     OrderDate DATE DEFAULT GETDATE(),
62     OrderStatus VARCHAR(50)
```

ER DIAGRAM:



Task 2(Sub Query)

1. Simple Subquery

A basic subquery used in a WHERE clause.

```
160 • SELECT ProductName, Category, UnitPrice
161 FROM Product
162 WHERE SupplierID IN (
163     SELECT SupplierID
164     FROM Supplier
165     WHERE City = 'Karachi'
166 );
```

Result Grid | Filter Rows: | Export:

	ProductName	Category	UnitPrice
▶	LED Bulb	Electronics	250.00
	Paint Brush Set	Hardware	1200.00

2. Single Row Subquery

A subquery that returns exactly one value.

```
169 • SELECT ProductName, UnitPrice FROM Product
170 WHERE UnitPrice = (
171     SELECT MAX(UnitPrice)
172     FROM Product
173 );
```

Result Grid | Filter Rows: | Export:

	ProductName	UnitPrice
▶	Electric Drill	7500.00

3. Multiple Row Subquery

A subquery that returns multiple values

```
176 • SELECT SupplierName, City FROM Supplier
177 WHERE SupplierID IN (
178     SELECT SupplierID
179     FROM Product
180     WHERE UnitPrice > 2000
181 );
```

Result Grid | Filter Rows: | Export:

	SupplierName	City
▶	Metro Traders	Lahore
	Prime Wholesales	Islamabad

4. Correlated Subquery

Subquery that references a column from the outer query.

```
184 • SELECT ProductName, UnitPrice, SupplierID FROM Product p
185 WHERE UnitPrice > (
186     SELECT AVG(UnitPrice)
187     FROM Product
188     WHERE SupplierID = p.SupplierID
189 );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

ProductName	UnitPrice	SupplierID
-------------	-----------	------------

5. Nested Subquery

A subquery inside another subquery.

```
192 • SELECT CustomerName FROM Customer
193 WHERE CustomerID IN (
194     SELECT CustomerID FROM Orders
195     WHERE OrderID IN (
196         SELECT OrderID FROM OrderDetails
197         WHERE ProductID IN (
198             SELECT ProductID FROM Product
199             WHERE SupplierID = (
200                 SELECT SupplierID FROM Supplier WHERE SupplierName = 'Metro Traders'
201             )
202         )
203     )
204 )
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

CustomerName
Zara Fatima

6. Subquery with INSERT

Insert new record based on result of subquery.

```
7 • INSERT INTO Shipment (OrderID, WarehouseID, ShipmentDate, DeliveryStatus)
3 VALUES (
3     (SELECT MAX(OrderID) FROM Orders),
3     1,
1     NOW(),
2     'Processing'
3 );
```

7. Subquery with UPDATE

Update rows using data from a subquery.

```
UPDATE Inventory
SET QuantityInStock = QuantityInStock + 20
WHERE ProductID IN (
    SELECT ProductID FROM Product
    WHERE SupplierID = (
        SELECT SupplierID FROM Supplier WHERE SupplierName = 'Global Supply Co'
    )
);
```

8. Subquery with DELETE

Delete records based on another table.

- ```
DELETE FROM Orders
WHERE CustomerID IN (
 SELECT CustomerID FROM Customer WHERE City = 'Multan'
);
```

## 9. Subqueries with IN

```
232 • SELECT ProductName FROM Product
233 WHERE ProductID IN (
234 SELECT ProductID FROM Inventory
235 WHERE WarehouseID IN (
236 SELECT WarehouseID FROM Warehouse
237 WHERE Location IN ('Lahore', 'Karachi')
238)
239);
240
241
```

|                                                                                                         |
|---------------------------------------------------------------------------------------------------------|
| <                                                                                                       |
| Result Grid                                                                                             |
| Filter Rows: <input type="text"/>                                                                       |
| Export:  Wrap Cell C |
| ProductName                                                                                             |
| LED Bulb                                                                                                |
| Ceiling Fan                                                                                             |

## Task 3(Joins)

### 1. INNER JOIN

Returns records where matching values exist in both tables.

```
242 • SELECT
243 p.ProductName,
244 p.Category,
245 p.UnitPrice,
246 s.SupplierName,
247 s.City FROM Product p
248 INNER JOIN Supplier s
249 ON p.SupplierID = s.SupplierID;
250
```

|   | ProductName     | Category    | UnitPrice | SupplierName     | City       |
|---|-----------------|-------------|-----------|------------------|------------|
| ▶ | LED Bulb        | Electronics | 250.00    | Global Supply Co | Karachi    |
|   | Ceiling Fan     | Electronics | 3500.00   | Metro Traders    | Lahore     |
|   | Electric Drill  | Hardware    | 7500.00   | Prime Wholesales | Islamabad  |
|   | Paint Brush Set | Hardware    | 1200.00   | Quick Logistics  | Karachi    |
|   | Screwdriver Kit | Tools       | 1800.00   | SupplyMart       | Faisalabad |

### 2. FULL OUTER JOIN (Simulated with UNION in MySQL)

MySQL doesn't support FULL OUTER JOIN directly — use **UNION** of LEFT + RIGHT joins.

```
252 • SELECT
253 s.SupplierName,
254 p.ProductName,
255 p.Category FROM Supplier s
256 LEFT JOIN Product p ON s.SupplierID = p.SupplierID
257 UNION
258 SELECT
259 s.SupplierName,
260 p.ProductName,
261 p.Category FROM Supplier s
262 RIGHT JOIN Product p ON s.SupplierID = p.SupplierID;
```

|   | SupplierName     | ProductName     | Category    |
|---|------------------|-----------------|-------------|
| ▶ | Global Supply Co | LED Bulb        | Electronics |
|   | Metro Traders    | Ceiling Fan     | Electronics |
|   | Prime Wholesales | Electric Drill  | Hardware    |
|   | Quick Logistics  | Paint Brush Set | Hardware    |
|   | SupplyMart       | Screwdriver Kit | Tools       |

### 3. LEFT OUTER JOIN

Returns all records from the **left table**, and matching ones from the right.

```
265 • SELECT
266 c.CustomerName,
267 c.City,
268 o.OrderID,
269 o.Status
270 FROM Customer c
271 LEFT JOIN Orders o
272 ON c.CustomerID = o.CustomerID;
273
```

Result Grid | Filter Rows: | Export

|   | CustomerName | City       | OrderID | Status    |
|---|--------------|------------|---------|-----------|
| ▶ | Hassan Ali   | Karachi    | 1       | Pending   |
|   | Zara Fatima  | Lahore     | 2       | Completed |
|   | Bilal Khan   | Islamabad  | 3       | Shipped   |
|   | Sana Ahmed   | Faisalabad | 4       | Pending   |
|   | Imran Abbas  | Multan     | 5       | Completed |

### 4. RIGHT OUTER JOIN

Returns all records from the **right table**, and matching ones from the left.

```
75 • SELECT
76 o.OrderID,
77 o.Status,
78 c.CustomerName,
79 c.City
80 FROM Customer c
81 RIGHT JOIN Orders o
82 ON c.CustomerID = o.CustomerID;
83
```

Result Grid | Filter Rows: | Export

| OrderID | Status    | CustomerName | City       |
|---------|-----------|--------------|------------|
| 1       | Pending   | Hassan Ali   | Karachi    |
| 2       | Completed | Zara Fatima  | Lahore     |
| 3       | Shipped   | Bilal Khan   | Islamabad  |
| 4       | Pending   | Sana Ahmed   | Faisalabad |
| 5       | Completed | Imran Abbas  | Multan     |

### 5. SELF JOIN

Join a table to itself — useful for hierarchical or comparative relationships.

```
285 • SELECT
286 s1.SupplierName AS Supplier1,
287 s2.SupplierName AS Supplier2,
288 s1.City
289 FROM Supplier s1
290 JOIN Supplier s2
291 ON s1.City = s2.City AND s1.SupplierID <> s2.SupplierID
292 ORDER BY s1.City;
293
```

Result Grid | Filter Rows: | Export | Wrap Cell Contents

| Supplier 1       | Supplier2        | City    |
|------------------|------------------|---------|
| Quick Logistics  | Global Supply Co | Karachi |
| Global Supply Co | Quick Logistics  | Karachi |

## Task 4(Indexes)

### 1. Create Simple Index

Used to improve search performance on frequently queried columns.

```
CREATE INDEX idx_product_name
ON Product(ProductName);
```

### 2. Create Unique Index

Ensures **no duplicate values** in the indexed column(s).

```
CREATE UNIQUE INDEX idx_customer_email
ON Customer(Email);
```

### 3. Create Composite Index

Combines multiple columns to optimize queries that use both in WHERE or ORDER BY.

```
CREATE INDEX idx_product_category_price
ON Product(Category, UnitPrice);
```

### 4. Drop Index

Removes an existing index when it's no longer needed or causing overhead.

```
DROP INDEX idx_product_name
ON Product;
```



## Task 5(Views)

### 1. Create View

Create a simple view showing product details with supplier names.

```
• CREATE VIEW vw_ProductDetails AS
SELECT
 p.ProductID,
 p.ProductName,
 p.Category,
 p.UnitPrice,
 s.SupplierName,
 s.City AS SupplierCity
FROM Product p
JOIN Supplier s ON p.SupplierID = s.SupplierID;
```

### 2. Update View

Modify an existing view to include additional columns or changes.

```
ALTER VIEW vw_ProductDetails AS
SELECT
 p.ProductID,
 p.ProductName,
 p.Category,
 p.UnitPrice,
 p.QuantityPerUnit,
 s.SupplierName,
 s.City AS SupplierCity
FROM Product p
JOIN Supplier s ON p.SupplierID = s.SupplierID;
```

### 3. Delete from View

You can delete records directly through an updatable view.

```
DELETE FROM vw_ProductDetails
WHERE ProductID = 5;
```

#### 4. Add Condition to View

Create a view with specific conditions for filtered records.

```
CREATE VIEW vw_AvailableProducts AS
SELECT
 p.ProductID,
 p.ProductName,
 p.UnitPrice,
 i.QuantityInStock
FROM Product p
JOIN Inventory i ON p.ProductID = i.ProductID
WHERE i.QuantityInStock > 0;
```

#### 5. Drop View

Remove a view when it's no longer needed.

```
DROP VIEW vw_ProductDetails;
```

#### 6. Dynamic (Standard) View

A **dynamic view** always shows *live* data from base tables.

```
CREATE VIEW vw_OrderSummary AS
SELECT
 o.OrderID,
 c.CustomerName,
 o.OrderDate,
 o.Status,
 SUM(od.Quantity * od.UnitPrice) AS TotalAmount
FROM Orders o
JOIN Customer c ON o.CustomerID = c.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY o.OrderID, c.CustomerName, o.OrderDate, o.Status;
```

#### 7. Static (Materialized) View (Simulated in MySQL)

MySQL doesn't have true **materialized views**, but you can simulate them using a **table snapshot**.

```
CREATE TABLE mv_OrderSnapshot AS
SELECT
 o.OrderID,
 c.CustomerName,
 o.OrderDate,
 SUM(od.Quantity * od.UnitPrice) AS TotalAmount
FROM Orders o
JOIN Customer c ON o.CustomerID = c.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY o.OrderID, c.CustomerName, o.OrderDate;
```

